



BINARY SEARCH

🎯 **GOAL:** Reduce search space by half in every step — $O(\log n)$ time

Page 1 of 4

FAANG INTERVIEW CHEAT SHEET (JAVA)

1. WHEN TO USE BINARY SEARCH?

- ✓ Data is sorted or can be converted to sorted.
- ✓ We can eliminate half of the search space.
- ✓ Answer looks like → F F F T T T (False → True)
- ✓ We can check a condition in $O(1)$ time.
- ✓ Used for exact match, first/last occurrence, or finding an answer in a search space.

💡 AHA MOMENT

Binary Search is **not** about searching an element — it's about finding the **boundary** where the answer changes.

F F F F T T T T

Find the **FIRST T** (the transition point).

2. CLASSIC BINARY SEARCH TEMPLATE (EXACT MATCH)

```
int left = 0, right = n - 1;
while (left <= right) {
    int mid = left + (right - left) / 2; // safe mid

    if (nums[mid] == target) {
        return mid; // found
    } else if (nums[mid] < target) {
        left = mid + 1; // search right half
    } else {
        right = mid - 1; // search left half
    }
}
return -1; // not found
```

💡 **Why $\text{mid} = \text{left} + (\text{right} - \text{left}) / 2$?**

To avoid integer overflow that can happen with $(\text{left} + \text{right}) / 2$.

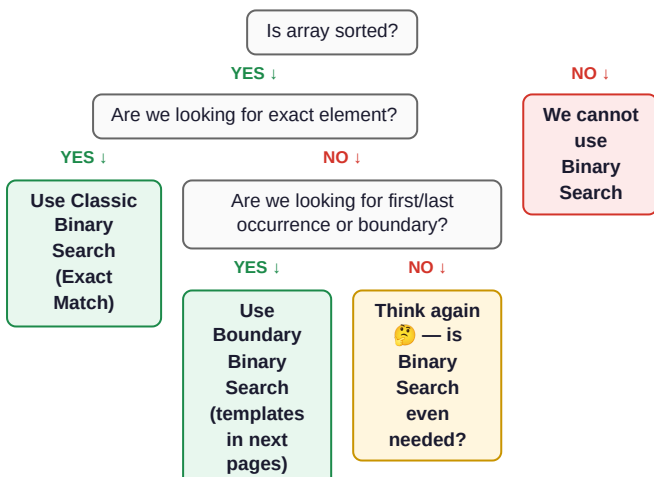
★ INTERVIEW TIP

- Always decide what the array is trying to tell you.
- Define the condition, not the element.
- Think: **What am I searching for?**
- Dry run on examples before coding.

✗ COMMON MISTAKES

- ✗ Wrong boundary update (leads to infinite loop).
- ✗ Using $\text{mid} = (\text{left} + \text{right}) / 2$ (overflow risk).
- ✗ Not handling edge cases (empty array, size 1).
- ✗ Wrong condition ($<$ vs $<=$).
- ✗ Returning wrong index (first/last occurrence problems).

3. BINARY SEARCH DECISION TREE



4. BINARY SEARCH ON SORTED ARRAY (EXAMPLE)

Array: [5, 7, 7, 8, 8, 10] Target = 8

Step 1: left=0, right=5, mid=2 → nums[2]=7 7<8, move right (left=3)

Step 2: left=3, right=5, mid=4 → nums[4]=8 8==8, found!

Step 3: Return index 4

5. COMPLEXITY ANALYSIS

Operation	Complexity
Time Complexity	$O(\log n)$
Space Complexity	$O(1)$ (Iterative)
Best Case	$O(1)$ (if mid is target)
Worst Case	$O(\log n)$

📌 **Remember:** Every step eliminates half of the search space. That's the power of Binary Search! 🧠

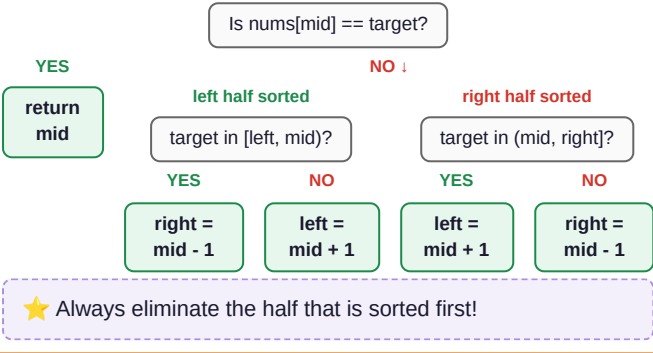


6. VARIANTS OF BINARY SEARCH

Variant	When to Use
Classic Binary Search	Exact match in sorted array
First Occurrence	Find first index of target
Last Occurrence	Find last index of target
Search in Rotated Sorted Array	Array is sorted but rotated
Search in Peak Element	Find peak element in bitonic array
Binary Search on Answer	Answer lies in a range, not searching an element

8. SEARCH IN ROTATED SORTED ARRAY (UNIQUE ELEMENTS)

Example: nums = [4,5,6,7,0,1,2], target = 0



10. VISUALIZING BINARY SEARCH (BOUNDARY SEARCH)

Find First Occurrence of target = 4 in nums = [1, 2, 4, 4, 4, 5, 6]



Step 1: left=0, right=6, mid=3 → nums[3]=4==4, ans=3, right=mid-1=2

Step 2: left=0, right=2, mid=1 → nums[1]=2, 2<4, left=mid+1=2

Step 3: left=2, right=2, mid=2 → nums[2]=4==4, ans=2, right=mid-1=1

left(2) > right(1) → loop stops

✓ Answer = 2 (first index of 4)

7. FIRST & LAST OCCURRENCE TEMPLATES

First Occurrence (Lower Bound)

```
int firstOccurrence(int[] nums, int target) {
    int left = 0, right = nums.length - 1;
    int ans = -1;
    while (left <= right) {
        int mid = left + (right - left) / 2;
        if (nums[mid] >= target) {
            if (nums[mid] == target) ans = mid;
            right = mid - 1; // move left
        } else {
            left = mid + 1; // move right
        }
    }
    return ans;
}
```

Last Occurrence (Upper Bound)

```
int lastOccurrence(int[] nums, int target) {
    int left = 0, right = nums.length - 1;
    int ans = -1;
    while (left <= right) {
        int mid = left + (right - left) / 2;
        if (nums[mid] <= target) {
            if (nums[mid] == target) ans = mid;
            left = mid + 1; // move right
        } else {
            right = mid - 1; // move left
        }
    }
}
```

9. BINARY SEARCH ON ANSWER

When to use?

- Answer is not an element from the array.
- We're asked to find a min/max value that satisfies a condition.
- The condition is monotonic (T...T...T or F...F...F).

```
int left = 0;
int right = (int)1e9;
int ans = -1;

while (left <= right) {
    int mid = left + (right - left) / 2;
    if (isValid(mid)) {
        ans = mid;
        right = mid - 1; // try smaller answer
    } else {
        left = mid + 1; // try bigger answer
    }
}
```

11. COMMON MISTAKES & HOW TO AVOID THEM

Mistake	Why It's Wrong	Fix
Using (left+right)/2	Can overflow when both are large	Use left + (right-left)/2
Incorrect boundary update	Infinite loop or wrong result	Be careful with left=mid+1 / right=mid-1
Not handling edge cases	Empty array, size 1, target not found	Always test edge cases
Confusing first & last occurrence	Logic differs in shrinking direction	First → shrink right; Last → shrink left



BINARY SEARCH

FAANG INTERVIEW CHEAT SHEET (JAVA)

GOAL: Reduce search space by half in every step — $O(\log n)$ time

Page 3 of 4

12. BOUNDARY SEARCH VISUALIZATION

We are looking for FIRST occurrence of 8

0	1	2	3	4	5	6	7	8	9	10
1	2	4	5	7	8	8	8			
8	10	13								

Step	left	right	mid	nums[mid]	Action
1	0	10	5	8	ans=5, move left (right=4)
2	0	4	2	4	nums[mid]<8, move right (left=3)
3	3	4	3	5	nums[mid]<8, move right (left=4)
4	4	4	4	7	nums[mid]<8, move right (left=5)
5	5	4	—	—	left > right, stop

✓ Answer = 5 (First index of 8)

💡 **Key Idea:** When $\text{nums}[\text{mid}] == \text{target}$, don't return immediately — store the answer and keep searching the left half.

14. BINARY SEARCH ON 2D MATRIX (ROW-WISE SORTED)

Example matrix (search = 34):

1	3	5	7
10	11	16	20
23	30	34	50
60	65	70	80

Approach: Treat matrix as 1D array. Index mapping: $\text{row} = \text{mid} / \text{cols}$; $\text{col} = \text{mid} \% \text{cols}$. Apply normal binary search.

```
int rows = matrix.length, cols = matrix[0].length;
int left = 0, right = rows * cols - 1;
while (left <= right) {
    int mid = left + (right - left) / 2;
    int r = mid / cols, c = mid % cols;
    if (matrix[r][c] == target) return true;
    else if (matrix[r][c] < target) left = mid + 1;
    else right = mid - 1;
}
return false;
```

★ Works because each row is sorted and the first element of a row is greater than the last element of the previous row.

16. SEARCH IN INFINITE / VERY LARGE ARRAY

Problem: Array is sorted but size is unknown or very large. Target = 15.

Solution: Find a range (low, high) where the target can exist, then apply binary search in that range.

```
// 1. Expand the range
low = high + 1;
high = high * 2;
// repeat until arr[high] >= target
// or arr[high] goes out of bounds
// 2. Apply normal binary search between low and high
```

✓ Time Complexity: $O(\log \text{index_of_target})$

13. MORE BINARY SEARCH TEMPLATES

Lower Bound (First index where $\text{nums}[i] \geq \text{target}$)

```
int left = 0, right = n;
while (left < right) {
    int mid = left + (right - left) / 2;
    if (nums[mid] >= target) right = mid;
    else left = mid + 1;
}
return left; // n if none exists
```

Use case: insertion position (upper bound logic), Longest Increasing Subsequence (LIS) **Upper Bound (First index where $\text{nums}[i] > \text{target}$)**

```
int left = 0, right = n;
while (left < right) {
    int mid = left + (right - left) / 2;
    if (nums[mid] > target) right = mid;
    else left = mid + 1;
}
return left; // n if none exists
```

Use case: count elements $\leq \text{target}$, find last occurrence + 1 **Floor (Last index where $\text{nums}[i] \leq \text{target}$)**

```
int left = 0, right = n - 1, ans = -1;
while (left <= right) {
    int mid = left + (right - left) / 2;
    if (nums[mid] <= target) {
        ans = mid; left = mid + 1;
    } else right = mid - 1;
}
```

15. SEARCH IN PEAK ELEMENT (BITONIC ARRAY)

Bitonic array (first increasing, then decreasing): [1, 3, 8, 12, 4, 2] — Search = 4

Steps: 1) Find the peak index. 2) If $\text{target} == \text{peak}$ → return true. 3) If $\text{target} < \text{peak}$ → search left (increasing) half. 4) If $\text{target} > \text{peak}$ → search right (decreasing) half.

Find Peak Index

```
int l = 0, r = n - 1;
while (l < r) {
    int mid = l + (r - l) / 2;
    if (arr[mid] < arr[mid + 1]) l = mid + 1;
    else r = mid;
}
return l;
```

Binary Search in Decreasing Order

```
while (l <= r) {
    int mid = l + (r - l) / 2;
    if (arr[mid] == target) return true;
    else if (arr[mid] > target) l = mid + 1;
    else r = mid - 1;
}
```

17. REAL INTERVIEW QUESTIONS (POPULAR)

- | | |
|--|--|
| 1. Binary Search | 7. Minimum Eating Speed |
| 2. Search Insert Position | 8. Ship Packages Within D Days |
| 3. Find First and Last Position of Element in Sorted Array | 9. Allocate Minimum Number of Pages |
| 4. Search in Rotated Sorted Array | 10. Split Array Largest Sum |
| 5. Search in Rotated Sorted Array II (duplicates) | 11. Find Peak Element |
| 6. Koko Eating Bananas | 12. Find Minimum in Rotated Sorted Array |

🏆 Most questions are variations of: Boundary Search · Search on Answer · Rotated Array Search



BINARY SEARCH

FAANG INTERVIEW CHEAT SHEET (JAVA)

GOAL: Reduce search space by half in every step — $O(\log n)$ time

Page 4 of 4

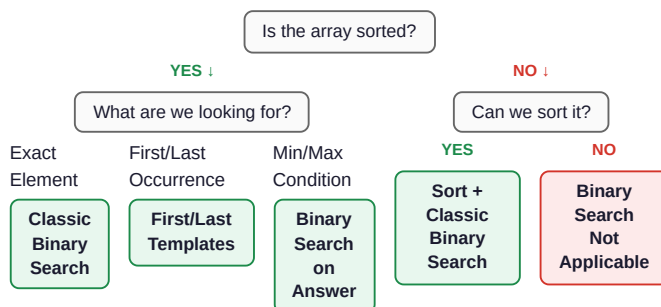
18. TIME & SPACE COMPLEXITY SUMMARY

Problem Type	Time	Space	Notes
Classic Binary Search	$O(\log n)$	$O(1)$	Iterative
First / Last Occurrence	$O(\log n)$	$O(1)$	Extra checks
Search on Answer	$O(\log N \times \log M)$	$O(1)$	N =array size, M =answer range
Search in 2D Matrix	$O(\log(m \times n))$	$O(1)$	Treat as 1D array
Peak Element	$O(\log n)$	$O(1)$	Bitonic array
Rotated Sorted Array	$O(\log n)$	$O(1)$	Unique elements

19. PATTERN RECOGNITION GUIDE

Look at the problem	Pattern
Exact value in sorted array	Classic Binary Search
First / Last position of a target	First / Last Occurrence Templates
Smallest / Largest value satisfying a condition	Binary Search on Answer (Optimization)
Matrix is sorted (row-wise & col-wise)	Treat as 1D Binary Search
Array is rotated sorted	Find pivot, then Binary Search in correct half
Bitonic / Mountain array	Find peak, then search in increasing/decreasing part

20. BINARY SEARCH FLOW SUMMARY



21. BINARY SEARCH SUMMARY (KEY POINTS)

- ✓ Always think: can I eliminate half of the search space?
- ✓ Define the search space correctly.
- ✓ Choose the right template (classic / first / last / answer).
- ✓ Handle boundaries and edge cases.
- ✓ Avoid infinite loops by correct pointer updates.
- ✓ Dry run with small examples before coding.
- ✓ Write clean, bug-free, well-commented code.

Remember: Binary Search is all about reducing uncertainty in the smartest way possible!

23. DRY RUN TIPS

- Always write left, right, mid.
- Highlight the mid element.
- Write the comparison result.
- Update the pointers.
- Repeat until the condition breaks.
- Finally, write the answer.

Pro Tip: Practice dry runs without coding first — it improves your problem-solving speed.

24. QUICK REFERENCE (JAVA)

Mid (Safe)	<code>left + (right - left) / 2</code>
Left Half	<code>right = mid - 1;</code>
Right Half	<code>left = mid + 1;</code>
Loop Condition	<code>while (left <= right)</code>
Return Not Found	<code>return -1;</code>
First Occurrence (on match)	<code>ans=mid; right = mid - 1;</code>
Last Occurrence (on match)	<code>ans=mid; left = mid + 1;</code>
Lower Bound (half-open)	<code>if(nums[mid]>=target) right=mid;</code>

25. FAANG INTERVIEW CHECKLIST

- ✓ Understand problem & constraints
- ✓ Identify if Binary Search applies
- ✓ Choose correct template
- ✓ Define search space
- ✓ Handle edge cases
- ✓ Dry run with examples
- ✓ Check time & space complexity
- ✓ Test with custom cases
- ✓ Explain your approach clearly
- ✓ Be ready for follow-up questions
- ✓ Optimize if required

26. MOTIVATION BOOSTER

- ★ Binary Search is a superpower.
- ★ It turns $O(n)$ into $O(\log n)$.
- ★ Master this pattern, crack any interview!

YOU'VE GOT THIS! 💪